PACT Compute teacher demonstration guide

Rehearsal instructions website walkthrough and viva preparation

13 November 2026

What you are demonstrating

Your demonstration should answer one question: how does a written agreement become a checked GPU execution with evidence? Begin with the agreement, show a feasible job running on the laptop GPU, inspect its output, and then show an infeasible request being rejected. Add the marketplace and research sections once that central story is clear. A 15 to 20 minute demonstration is enough; keep deeper experiments for questions.

Use this opening explanation: “Our project connects service agreements, GPU availability, job execution, and independent auditing. A reviewer approves supported rules extracted from an agreement. The system checks those rules before allocating hardware, runs an image embedding workload, and records evidence. We also compare scheduling and pricing methods in a separate research environment.”

Do not describe the system as training an image classifier, supporting arbitrary programs, implementing real payments, or proving commercial collusion. The live workload extracts 960-dimensional image embeddings using a pretrained model. The research scheduler and pricing experiments are separate from the actual laptop GPU execution.

Start the complete local demonstration

Open PowerShell in the project root, the folder containing run-demo.cmd. Execute this command:

```
.\run-demo.cmd
```

The launcher builds the frontend, prepares the local demo database and accounts, imports the dataset, generates publication results if absent, checks the model and GPU, and starts the API, worker, and runner. It opens http://127.0.0.1:8767. First-time preparation can take several minutes and may download dependencies or model weights. Prepare it before the class rather than during your introduction.

To run from any PowerShell location on this laptop, use:

```
& 'C:\Users\raddo\Documents\main el 5th sem\run-demo.cmd'
```

To stop only the services started for this demonstration:

```
.\run-demo.cmd -Stop
```

The stop command retains saved accounts, agreements, research results, and artifacts. Do not delete the database to clear the screen. Use a new agreement name or job name during each rehearsal. If the launcher says port 8767 is occupied, inspect the already-running demo or stop it with the command above before restarting.

The optional -NoBrowser switch starts services without opening a browser. The -NoGpu switch starts the API and research worker without launching a physical runner; use it only for non-execution demonstrations. The -Refresh switch reinstalls the demo runtime dependencies and synchronizes frontend packages before building. A successful -NoGpu launch is not a successful GPU test.

This demo is intentionally separate from the older server at port 8765. Its database, credentials, logs, and outputs live in pact-compute/.local/faculty-demo. The older server uses the team's PostgreSQL

configuration. Do not mix account passwords or evidence files between these two installations. The new demo does not require a Groq key or a cloud database.

Find credentials without exposing secrets

For the administrator walkthrough, sign in with username admin and the admin_password value in pact-compute/.local/faculty-demo/local-credentials.json. Open the file privately before sharing your screen. It also contains runner and signing secrets, so never show the whole file to the teacher, paste it into slides, or upload it with the report.

Marketplace role passwords are in pact-compute/.local/faculty-demo/market-credentials.json. The usernames are demo-producer, demo-consumer, demo-consumer-2, demo-reviewer, and demo-reviewer-2. The two reviewers belong to the corresponding consumer workspaces. These accounts are generated for a local demonstration, not public deployment.

Use different browser profiles or different browser applications for producer and consumer sessions. Several private windows in the same browser can share a private session, so merely opening another private window may not isolate the accounts. A convenient arrangement is Edge for the administrator, Chrome for the consumer, and a separate Edge profile for the producer.

Check readiness before class

Connect the laptop charger and keep it awake. Close unrelated GPU-heavy applications. Start the launcher, sign in, and confirm the console shows GPU connected. This laptop reports an NVIDIA GeForce RTX 4050 Laptop GPU with approximately 6 GiB of memory. The example 4 GiB agreement is feasible; the 32 GiB rejection example is deliberately infeasible.

Confirm that Research Lab has an imported snapshot and that Explainable AI displays Publication comparisons. Submit one small rehearsal job with built-in images and confirm that its Output download appears. A healthy API response alone is not sufficient. The runner log is pact-compute/.local/faculty-demo/runner.log; the API and worker logs are in the same directory.

Keep a copy of a successful output artifact and evidence export from your own rehearsal. If hardware or internet becomes unavailable during class, clearly label these as saved results. Do not present an old screenshot as a new execution. The core local workflow can operate without model-provider access once dependencies, data, and model weights are prepared.

Demonstration schedule

Segment	Suggested time	What the teacher should learn
Architecture and overview	2 minutes	API worker and runner have separate roles
Agreement and approval	3 minutes	Text becomes reviewed source-linked rules
Successful live job	3 minutes	Real GPU execution produces a verifiable artifact
Rejection and ambiguity	2 minutes	Valid policies can be infeasible and unclear terms block approval
Marketplace	4 minutes	Quotes reservations and shared receipts preserve isolation
Research and limitations	4 minutes	Metrics show trade-offs rather than a universal accuracy score

Step one explain the overview

Sign in as admin and open Overview. Point out the GPU connection and the current jobs or activity. Explain that the browser does not directly run GPU code. It sends requests to FastAPI, which stores decisions in the database. A separate runner receives authorized work and sends results. Research experiments are processed by a separate worker.

Say: “The green API health status checks database access. The GPU connected indicator comes from runner activity. These are separate readiness checks.” This prepares you to answer why the website might load while a job cannot run.

The administrator account combines permissions for a compact demonstration. Explain that ordinary consumers cannot approve their own policies. You will demonstrate separate roles in the marketplace or invitation workflow if time permits.

Step two create and approve an agreement

Open Agreements. Choose Load example. Inspect the service agreement text and give it a memorable name such as Faculty demo SLA. Click Extract rules. The expected result is six source-linked rules. Point out the memory, deadline, charge, maximum deferral, carbon preference, and reporting clauses.

Explain unit conversion with one example: 4 GiB becomes 4,096 MiB, while 12 minutes becomes 720 seconds. The original text remains available so the reviewer can inspect why a rule was generated. Review the extracted values, then click Approve policy. The agreement should show Approved.

Say: “Extraction proposes a structured policy. Approval is a separate step. We preserve the source and revision so later decisions can be traced to the exact agreement.” If asked whether this is a general legal-language engine, explain that the local parser supports bounded rule patterns and explicitly flags ambiguity or unsupported clauses.

Step three execute a real GPU job

Open Submit a job. Enter a job name such as Faculty live image embeddings. Select the agreement you just approved. Keep the four built-in images for the first presentation; this avoids file-format problems and

makes the demonstration repeatable. You can show uploads afterward. Supported uploads are PNG, JPEG, and WebP, with up to sixteen images.

Click Preview placement. The expected message is “An eligible reservation is available.” Inspect the selected runner and the planned terms. Explain that preview checks the request against current policy and inventory before execution. Then click Submit live job.

On the job detail page, observe the lifecycle and wait for Output to appear. Do not repeatedly resubmit because a job takes a few seconds. Explain that startup and input processing are part of the full execution path. The live reservation estimate is 38 plus three times the image count in seconds; it differs from GPU inference time.

Open or download Output. Show that the artifact contains four rows and a 960-value embedding per image. Each row includes an input hash. Explain: “The output is a feature vector, not a class label. We are demonstrating policy-controlled execution and verifiable output, not claiming classification accuracy.”

Inspect Policy audit and the execution evidence. Download the signed evidence if the teacher wants an artifact. Explain that hashes detect changes in bytes and the HMAC verifies the exported manifest using the installation's secret. Never disclose that secret as part of the demonstration.

Step four show a correct rejection

Return to Agreements and create a new agreement named Impossible memory example. Enter “Jobs require at least 32 GiB VRAM.” Extract and approve it. The text is clear and structurally valid, so approval is reasonable even though this laptop cannot satisfy it.

Open Submit a job, select this agreement, and click Preview placement. Expect “No runner satisfies the current policy and inventory.” Point to the minimum-memory constraint and actual runner capacity. Click Record rejected request to retain the decision in the job history.

Say: “Approval means the policy is understood and authorized. Admission means available resources can satisfy it. The project distinguishes these two decisions.” This is one of the strongest examples because it shows the system preventing an unsuitable execution rather than merely displaying a successful animation.

For the ambiguity example, create a separate draft containing “Jobs must finish by 17:00.” Extract the rules and show the unresolved date/timezone problem. Do not force approval or substitute a made-up timezone. Explain that an absolute deadline must identify when 17:00 actually occurs.

Step five demonstrate the exchange

Sign in as demo-producer in the Producer workspace using a separate browser session. Open GPU Exchange. Choose Publish capacity, select the enrolled local runner if prompted, enter a unique offer name, and choose Publish offer. Keep the default feasible availability window and price for your first rehearsal.

In the consumer session, sign in as demo-consumer in the Consumer workspace. Open GPU Exchange, find the offer by its exact name, and click its Request quote button. Select the preapproved image batch agreement, choose Get quote, and inspect the maximum charge and expiry. Explain that the quote is tentative and does not itself reserve the slot.

Choose Confirm reservation. Open Bookings on both sides. The producer and consumer should see the same booking progress and eventual result. Walk through accepted quote, lease, execution, and capacity release. Download the shared receipt on each side. Explain that both parties receive common booking evidence, while the consumer retains private access to its job inputs and embedding output.

The two-consumer race is optional because timing it manually can distract from the presentation. To rehearse it, open demo-consumer-2 in a third isolated session. Request two quotes for the same available offer before confirming either. Confirm both quickly. One consumer should obtain the reservation; the other should receive a message that the quoted slot or terms changed. Do not promise which consumer will win.

If asked why two quotes can exist for one GPU, answer: “A quote is an expiring proposal. Confirmation is the locked reservation step. The system prevents overlapping confirmed bookings, rather than forbidding everyone else from viewing a quote.”

Step six demonstrate team review

For a longer presentation, create a fresh Consumer account using Create account. Open Team and create an invitation code for a reviewer. In a separate browser profile, choose I have an invitation code, enter it, and create the reviewer account. Use a private rehearsal password rather than displaying a real personal password.

Create an agreement draft in the consumer workspace. Open the reviewer session and inspect the review inbox. Approve the draft there, then refresh the consumer view to show that it has become usable. Explain that the backend enforces the permission boundary; the consumer cannot approve merely by discovering the approval endpoint.

This optional segment demonstrates collaboration and organization isolation. It is useful if the teacher asks how the project works for a real team rather than one administrator. Keep the basic agreement-to-execution walkthrough first, since account setup consumes presentation time.

Step seven run the research experiment

In the admin session, open Research Lab. Select the imported snapshot. Use a run name such as Faculty scheduler comparison, set GPU jobs per seed to 45, and Paired seeds to 1 for a quick interactive example. Choose Run experiment and wait for Scheduler comparison and Provider strategy audit.

Show the five arms and compare completions, waiting time, overruns, and modeled carbon. Move the market round slider and inspect an automaton transition. Download the metrics CSV or evidence JSON. Explain that every arm sees the same generated arrivals and offer tape for a paired seed, which makes comparisons more meaningful.

Say: “This small interactive run demonstrates the mechanism. The publication table uses 20 paired seeds and 90 jobs per seed. Its exact numbers should not be expected from this one-seed example.” If you change supply pressure or controlled undercuts, explain that you changed the experimental condition before interpreting the new result.

Keep measured and modeled quantities separate. The Alibaba trace supplies workload observations, NESO supplies British grid history, and the replay models offers, durations, and energy. Do not claim that the graph measures this Indian laptop's actual carbon footprint or proves a real-world emissions reduction.

Step eight explain the research metrics

Open Explainable AI and Publication comparisons. Under Pricing and FSM, show the Q-table, intervention view, and learned pricing states. Under Runtime and allocation, show the runtime estimates and scheduler comparison. The report gives the full methodology; use the following compact answers during the demonstration.

Result	Number to remember	Qualification to say aloud
Rule extraction	180 of 180 exact matches	Supported template regression only
Temporal monitor	72 of 72 verdicts and zero unsafe accepts	Controlled trace fixtures
Pricing detector	F1 1.000	120 controlled labelled episodes
FSM action prediction	79.31 percent accuracy	Majority baseline 74.94 percent
Runtime bound	100 percent coverage	Excessively wide with worse pinball loss
Carbon aware versus policy only	About 169 g less modeled CO2	About 27 minutes more p95 wait

There is no single project-wide accuracy percentage. Extraction, temporal monitoring, price-response detection, action prediction, and runtime estimation are different tasks. Their scores use different denominators and cannot be averaged into one meaningful accuracy figure.

The runtime negative result is worth presenting confidently: “The bound covers all 46 test proxies but is too conservative. Its pinball loss is about 6,934 seconds, worse than the p90 baseline's 2,221 seconds. We therefore do not claim the calibrated predictor is ready for live scheduling.” Honest negative results demonstrate understanding.

Common viva questions

What is the main contribution? The integration of reviewed agreement constraints, admission and dispatch checks, actual GPU execution, organization-aware marketplace transactions, and independently inspectable evidence. Avoid claiming that every underlying algorithm is newly invented.

Why use a finite state machine? It provides states and transitions that can summarize and explain sequential behavior. The research FSM predicts pricing actions, while the protocol monitor checks execution ordering. These are related uses of sequential reasoning but are not the same component.

Why use CP-SAT? It represents discrete allocation decisions and constraints explicitly and searches for feasible optimized schedules. The research system compares its outcomes against simpler baselines so that added complexity can be evaluated rather than assumed beneficial.

Why not trust the language model? Model output can omit, invent, or misinterpret conditions. PACT requires source verification and review before enforcement. The local parser and deterministic guidance remain usable without model-provider access.

Does a CPU PyTorch wheel mean the job runs on the CPU? Not for the configured driver engine. PyTorch loads and preprocesses the model, while the native CUDA Driver API path executes GPU kernels. The preparation probe and returned CUDA artifact are the relevant checks.

Does 100 percent mean it always works? No. A perfect score applies to the specified finite test set. Template cases, controlled traces, and synthetic pricing episodes do not cover every real contract or market. State the dataset size and scope whenever quoting the score.

Is the HMAC a blockchain or public signature? No. It authenticates the manifest using a shared installation secret. Anyone holding that secret can generate a valid HMAC. It provides integrity within that trust model, not public nonrepudiation.

Is it ready for commercial deployment? It has a functioning implementation and local demonstration path, but production requires verified durable storage, recovery, monitoring, deployment identity, independent review, and a real workload pilot. Credits are not implemented payment settlement.

Deploying the project

The existing hosted address is https://pact-compute-sasha.onrender.com. Its health endpoint returned HTTP 200 with PostgreSQL during this review after a cold-start retry. That verifies reachability and database health at the check time. It does not verify that the deployed source equals this checkout or that a remote GPU transaction works.

For a new deployment, use the repository root as the Docker build context because the Dockerfile copies both pact-compute and PACT-Compute-data-preflight. The frontend is built into the API image. The API and worker run in the image; the physical GPU runner runs separately. Docker is not installed on this laptop, so the image build is an instruction to execute on a machine with an engine, not a completed check here.

```
docker build -t pact-compute:review .
```

In the existing Render service, verify the repository, intended branch, Dockerfile path, and exact commit before deploying. Configure PACT_DATABASE_URL for the approved PostgreSQL database, PACT_DB_TLS=1, PACT_URL for the HTTPS origin, PACT_SECURE_COOKIE=1, and stable PACT_SIGNING_SECRET and bootstrap credentials through the provider's secret settings. Keep real values out of source files and this guide.

Configure persistent artifact storage through an approved disk or the S3 backend. For S3, configure PACT_ARTIFACT_BACKEND=s3, PACT_S3_BUCKET, PACT_S3_PREFIX, and the required scoped AWS credentials or endpoint settings. Back up the database, artifacts, and signing secret as a matched set. Render's free-service local filesystem is ephemeral, so PostgreSQL alone does not preserve uploaded or generated artifact bytes.

Before migrating an existing shared database, take and verify the matched backup, stop older writers, and review migration head 0007. Run a dedicated Alembic upgrade step on the deployment host, then avoid competing migrations from multiple replicas. The container's PACT_RUN_MIGRATIONS setting controls startup migration behavior. Do not point experimental test commands at the team's real database.

Enroll the GPU host using the application's enroll-runner command with the correct organization. On that authorized GPU machine, configure PACT_URL to the hosted HTTPS origin, PACT_RUNNER_ID and PACT_RUNNER_TOKEN from enrollment, and the prepared GPU Python runtime. Start the runner with PYTHONPATH=backend and python -m pact.runner. Assign its real physical grid zone only if supported and verified.

After deployment, test login, reviewer approval, data availability, a real GPU job, output integrity, organization isolation, shared receipts, and artifact survival after restart. Check the worker separately from API health. Record the deployed commit and image identity. The full ordered checklist is in DEPLOYMENT.md and reports/release-candidate/HOSTED_STAGING_VERIFICATION.md. This review did not push code or redeploy the existing service.

Troubleshooting during rehearsal

If login fails, first confirm that you opened port 8767 and used the faculty-demo credential file. Account passwords from the older installation will not necessarily match. If the browser keeps showing another role, use a genuinely isolated browser profile or sign out before switching.

If GPU connected is absent, inspect runner.log and confirm the NVIDIA driver is available. A NoGpu launch intentionally has no physical runner. If the model preparation step reports a hash mismatch or CUDA error, stop and fix that issue before claiming successful execution. Do not fabricate a ready runner entry.

If a job is rejected, inspect its agreement, memory requirement, availability window, deadline, and charge limit. The 32 GiB example is supposed to fail. If a quote expires or another consumer books first, request a new quote against current availability. A changed-slot response is an expected contention outcome.

If Research Lab stays queued, inspect worker.log. If publication panels are missing, confirm the initial study generation finished. If the host initially times out, allow for a cold start and retry; persistent failure needs service logs and database checks. Do not assume that a sleeping service and a broken deployment are the same condition.

If Windows reports Application Control blocking a DLL in the new Python environment, use the tested application environment on this laptop and preserve security settings. A fresh machine may require its administrator to approve supported software or provide a compatible runtime. The clean Python 3.12 application setup was not fully validated here because of that block.

End by returning to the successful job and its evidence. Your closing sentence can be: “The demonstration shows an agreement becoming a reviewed policy, a real execution that satisfies the available constraints, and evidence that can be inspected afterward. The research results quantify trade-offs and identify where further validation is needed.”